

Especificação Técnica Definitiva:

1. Visão Geral do Projeto

Um aplicativo web (PWA) focado na leitura, marcação (grifos) e edição leve de documentos HTML pesados (leis, apostilas, Vade Mecum). O sistema é projetado para uso pessoal, operando perfeitamente em Desktops e Tablets (especialmente iPad). O grande diferencial é a **sincronização direta com o OneDrive** e a capacidade de exportar arquivos HTML 100% portáteis (standalone).

2. Stack Tecnológico (Moderno, Open-Source e Gratuito)

Para cumprir a premissa de "zero bugs" e alta estabilidade, a escolha das ferramentas baseia-se em bibliotecas modernas, de código aberto e gratuitas para uso pessoal.

- **Framework Front-end: React.js com TypeScript.** O TypeScript é inegociável aqui; ele garantirá o comportamento 100% determinístico exigido, evitando erros de tipagem.
- **Motor de Leitura/Edição (O Coração): Tiptap** (baseado no ProseMirror). Manipular o DOM puro no Safari/iPad é a maior fonte de bugs. O Tiptap mantém um modelo de dados próprio (JSON AST) sincronizado com o DOM, resolvendo nativamente o problema de grifar múltiplos blocos sem quebrar o HTML.
- **Posicionamento de Popovers: Floating UI.** Garante que o menu de cores do grifo nunca vaze da tela ou fique escondido, calculando colisões automaticamente.
- **Gerenciamento de Estado: Zustand.** Moderno, leve, sem boilerplate. Perfeito para gerenciar o estado do "Modo Grifar", preferências e status de sincronização.
- **Estilização: Tailwind CSS** combinado com variáveis CSS nativas para a troca dinâmica de temas e tipografia.
- **Higienização (Segurança): DOMPurify.** Limpa o HTML importado para evitar quebra de layout ou scripts indesejados.
- **Autenticação e Nuvem: @azure/msal-browser** (Microsoft Authentication Library) e **Microsoft Graph API.** Gratuitos para uso pessoal, gerenciam o login e a comunicação com o OneDrive.

3. Arquitetura e Fluxo de Dados

3.1. Sincronização Bidirecional (Leitura ↔ Edição)

Como usaremos o mesmo motor (Tiptap) para ambos os modos, a "sincronização" deixa de ser um problema de conversão de HTML e passa a ser apenas uma mudança de estado.

- **Modo Leitura:** O motor aplica uma *Mark* (tag `<mark>`) com um atributo de cor. A propriedade do editor fica `editable={false}`.
- **Modo Edição:** O usuário clica em editar. O motor muda para `editable={true}`. O HTML interno permanece intacto, destravando a digitação.

3.2. Fluxo de Sincronização com OneDrive (Cloud-First)

1. **Abertura:** O usuário faz login. O app usa a Graph API para listar e selecionar o arquivo .html no OneDrive. O app baixa o conteúdo e salva o itemId (ID único do arquivo) na memória.
2. **Edição:** O usuário faz os grifos (salvando temporariamente no LocalStorage/IndexedDB como backup).
3. **Salvamento (Sync):** Ao clicar em "Salvar", o app captura o HTML atual, pega o itemId e faz uma requisição PUT na Graph API, sobrescrevendo o arquivo original na nuvem silenciosamente.

3.3. Persistência Local (Fallback)

- leitor_html_preferences: Guarda Tema, Tamanho da Fonte, Família da Fonte e Largura da Tela.
- leitor_html_document: Guarda o conteúdo atual e o itemId para que, se a aba for fechada ou a internet cair, o usuário volte exatamente de onde parou.

4. Especificação de Funcionalidades (UI/UX)

Módulo 1: Header & Navegação

- **Responsividade:** Desktop exibe todos os botões. Mobile/iPad agrupa ações secundárias em um menu Dropdown.
- **Feedback Visual:** O botão "Modo Grifar" possui estado de *Toggle* (pressionado/solto) com cor de destaque e um *badge* numérico mostrando o total de grifos ativos.

Módulo 2: O Motor de Grifo (Modo Leitura)

- **Comportamento:** Ao selecionar um texto e soltar o dedo/mouse, o sistema intercepta a seleção, envelope o texto em `<mark class="hl-[cor]">` e limpa a seleção nativa do navegador.
- **Paleta Flutuante (Popover):** Acionada por clique/toque em qualquer elemento `<mark>`.
- **Comportamento Multibloco:** Grifos que cruzam múltiplos parágrafos são tratados como uma entidade conectada. Mudar a cor na paleta altera todos os blocos simultaneamente.
- **Cores Disponíveis:** Amarelo, Azul, Rosa, Verde, Roxo, Laranja, Cinza (todas em tons pastel via classes CSS).

Módulo 3: Aparência & Tipografia

Injeção de variáveis CSS no `:root`:

- **Tema:** Claro / Escuro.
- **Tamanho da Fonte:** Botões A- e A+ (ex: 14px a 24px).
- **Família da Fonte:** Alternância entre Sans-serif (Inter) e Serif (Merriweather).
- **Largura do Texto:** Estreita (65ch), Normal (80ch), Larga (100ch) e Total (100%).

Módulo 4: Integração OneDrive

- **Botão "Conectar OneDrive":** Abre pop-up seguro da Microsoft. Após login, mostra a foto do usuário.

- **Seletor de Arquivos:** Modal que lista pastas do OneDrive, filtrando apenas arquivos .html / .htm.
- **Botão "Salvar na Nuvem":** Com estados visuais ("Salvar", "Salvando...", "Salvo com sucesso").

Módulo 5: Exportação Standalone (Backup Local)

A função "Baixar HTML" gera um arquivo 100% portátil:

1. Captura o HTML atual do editor.
2. Injeta um bloco `<style>` no `<head>` com as regras exatas das cores dos grifos.
3. Gera um Blob (text/html) e força o download via JavaScript.

5. Requisitos Não-Funcionais e Tratamento de Riscos (Edge Cases)

5.1. PWA (Progressive Web App) e Resiliência Offline

- **Instalação:** O app deve ter um *Manifest* para ser "Instalado" na tela inicial do iPad, rodando em tela cheia sem a barra do Safari.
- **Conflito Offline vs. Nuvem:** Se o usuário clicar em "Salvar no OneDrive" sem internet, o app salva no IndexedDB e exibe: *"Sem conexão. Alterações salvas localmente. Sincronizaremos automaticamente quando você estiver online."* O *Service Worker* dispara o upload assim que a rede voltar.

5.2. Performance com Documentos Gigantes

- **Risco:** Leis com +500 páginas travando o navegador.
- **Solução:** O Tiptap é otimizado para isso, mas a renderização do React deve ser monitorada. O uso de paginação virtual (Virtualization) pode ser evitado se o DOM for mantido limpo, mas testes de estresse são obrigatórios.

5.3. Tratamento de Imagens no HTML

- **Solução:** O DOMPurify e o Tiptap devem ser configurados explicitamente para aceitar a tag `<img>`. Imagens em Base64 ou links externos devem ser mantidas intactas na importação, leitura, edição e exportação.

5.4. O Problema do "Longo Toque" no iOS (iPad/iPhone)

- **Risco:** Segurar o dedo aciona a lupa nativa ou o menu "Copiar/Pesquisar" da Apple.
- **Solução:** Aplicar `CSS user-select: text; -webkit-user-select: text;`. Com o "Modo Grifar" ativo, suprimir o menu nativo via JavaScript (`contextmenu.preventDefault()`).

5.5. Grifos Sobrepostos e Higienização

- **Risco:** Grifar uma frase que já contém uma palavra grifada gerando HTML quebrado.
- **Solução:** O Tiptap usará a função *merge marks*. A nova cor sobrescreve a antiga na intersecção, evitando aninhamento infinito de tags (`<mark><mark>`).

5.6. Expiração do Token do OneDrive

- **Risco:** Tokens da Microsoft expiram em 1 hora. O salvamento falharia após longo tempo de leitura.
 - **Solução:** A biblioteca @azure/msal-browser fará o *Silent Token Refresh*. Antes de salvar, o app verifica a validade do token e o renova em segundo plano sem interromper o usuário.
-

6. Fases de Desenvolvimento (Roadmap)

- **Fase 1: Fundação, UI Shell e PWA**
 - Setup: React, TypeScript, Tailwind, Zustand.
 - Criação da interface (Header, menus de aparência).
 - Configuração do *Service Worker* e *Manifest* (PWA).
- **Fase 2: O Motor de Leitura e Grifo (Core)**
 - Implementação do Tiptap em modo *readonly*.
 - Lógica de seleção de texto, aplicação da tag `<mark>` e *merge marks*.
 - Criação da paleta flutuante (Floating UI).
- **Fase 3: Modo Edição e Importação Segura**
 - Habilitar transição para o modo WYSIWYG (Edição).
 - Implementar upload local de arquivos HTML passando pelo DOMPurify (suporte a imagens).
 - Lógica de injeção de CSS para o download do HTML *standalone*.
- **Fase 4: Integração Cloud (OneDrive)**
 - Registro da aplicação no portal Microsoft Entra ID (Azure AD).
 - Implementação do login via MSAL e do seletor de arquivos (Graph API).
 - Criação da lógica de sobrescrita de arquivos (PUT request).
- **Fase 5: Resiliência Offline e Polimento**
 - Implementação da fila de sincronização offline (salvar localmente se não houver rede).
 - Testes de estresse com documentos gigantes.
 - Testes exaustivos no iPad (Safari) para validar bloqueio de menus nativos.